

Tek Şeritli Koridorlarda Çok Etmenli Yol Bulma için Kural ve Öncelik Tabanlı Melez CA* Algoritması

Ahmet Emin KIZILKAN

Özet

Amacı: Dar koridor ve tek şeritli endüstriyel topolojilerde görev yapan otonom yönlendirmeli araçların (AGV) yönlendirilmesi sırasında karşılaşılan koridor kilitlenmeleri ve darboğaz problemlerini çözerek toplam tamamlanma süresini optimize etmektir. **Araştırma Yöntemi:** Araştırmada, uzay-zaman yer ayırma tabanlı İşbirlikçi A* algoritması üzerinde kural ve öncelik tabanlı özgün bir melez model kurgulanmıştır. Model kapsamında araçların önceliklerini, konumlarını ve iş yüklerini içeren sözlüksel öncelik vektörü tanımlanmış, ana yolda bekleme tamamen yasaklanarak sıkı kısıtlı cepler ve depolar kullanılmıştır. Gereksiz manevraları önlemek için asimetrik maliyet yapısı uygulanmış ve araç içi görevler En Kısa İş İlk (SJF) yaklaşımıyla sıralanmıştır. **Bulgular:** Yapılan senaryo analizlerinde, fiziksel güvenlik mesafesi ihlalleri dışında algoritmik bir kilitlenme veya çarpışma hatasıyla karşılaşılmemiştir. Araç sayısı 5'ten 42'ye çıkarıldığında toplam zaman adımı doğrusal artış gösterirken, hesaplama süresinin üstel büyüdüğü ve 42 araçta 3.85 saniyeye ulaştığı gözlemlenmiştir. **Sonuç ve Öneriler:** Önerilen melez modelin dar alanlarda deterministik ve kararlı bir yönlendirme sağladığı, hesaplama süresinin çevrimdışı planlama için uygun olduğu sonucuna varılmıştır. Gelecek çalışmalarda hesaplama maliyetini düşürmek için bölgesel planlama ve dinamik zaman pencereleri yaklaşımlarının uygulanması önerilmektedir.

Anahtar Kelimeler: Otonom Yönlendirmeli Araçlar (AGV), İşbirlikçi A* (Cooperative A*), Kilitlenme Çözümü, Çoklu Ajanlı Yol Bulma (MAPF)

1 Giriş

Otonom Yönlendirmeli Araçların (AGV) endüstriyel tesislerde ve modern depo sistemlerinde kullanımını hızla yaygınlaştırmaktadır. Paylaşımlı alanlarda çok sayıda aracın eş güdümlü, güvenli ve verimli bir şekilde hareket etmesi, operasyonel sürekliliğin ve lojistik akışın iyileştirilmesi açısından kritik bir öneme sahiptir. Çok Etmenli Yol Bulma (MAPF) olarak adlandırılan bu problem temel olarak, etmenlerin birbirleriyle çarpışmadan ve kilitlenmeden (deadlock) başlangıç noktalarından hedeflerine ulaşmalarını amaçlamaktadır. Ancak sistemdeki araç sayısı ve harita karmaşıklığı arttıkça uzay ve zaman kısıtları altında optimal güzergahların hesaplanması zorlaşmakta ve problem genellikle NP-Zor bir yapıya dönüşmektedir.

Alan tasarrufu ve yapısal zorunluluklar nedeniyle dar koridorlar ve tek şeritli yol hatları sıklıkla tercih edilmektedir. Bu tür kısıtlı topolojiler, araçların yönlendirilmesi problemini çok daha kritik hale getirmektedir. Çünkü araçların birbirine yol verebileceği veya dinamik olarak güzergah değiştirebileceği alanlar mevcut değildir. Tek şeritli bir hat üzerinde zıt yönlerde hareket eden araçların karşı karşıya gelmesi (zıt yön çatışması [Bölüm 4.1](#)), tüm araçların tek bir istasyona yoğunlaşması (darboğaz [Bölüm 4.2](#)) ve kaçınma noktası bulamayan araçların oluşturduğu koridor kilitlenmeleri ([Şekil 1](#)), sistemin tamamen durmasına ve kayıplara yol açabilmektedir. Dolayısıyla bu kısıtlar altında fiziksel güvenlik mesafelerini koruyarak deterministik çözümler üretebilen yönlendirme mekanizmalarına ihtiyaç duyulmaktadır.

Literatürde MAPF problemlerinin gerçek zamanlı uygulanabilirliği ve ölçeklenebilirliği için bütüncül (*centralized*) çözümler yerine ayrık (*decoupled*) ve öncelik tabanlı yaklaşımlar sıklıkla tercih edilmektedir. Bu yaklaşımların en temel taşlarından biri olan İşbirlikçi A* (*Cooperative A**) algoritması etmenlerin hareketlerini eş güdümlü hale getirmek için uzay-zaman yer ayırma tablosu (*Space-Time Reservation Table*) kullanmakta ve dinamik engellerden kaçınma işlemini üç boyutlu bir arama problemine indirgemektedir. Ancak klasik CA* algoritması geniş ızgara tabanlı haritalarda başarılı sonuçlar verse de dar koridorlar ve tek şeritli topolojilerde ciddi iki zafiyete sahiptir. Birincisi eşit öncelikli etmenlerin çizelgeleme sırasının çözüme doğrudan etki etmesidir. İkinci ve daha önemli olanıysa dar alanlarda kaçacak yer bulamayan etmenlerin oluşturduğu koridor kilitlenmeleridir. Ayrıca klasik yaklaşım, araçların güzergahını uzatan gereksiz ileri-geri (*yo-yo*) hareketlerine zorlayabilmektedir.

Bu çalışmada, söz konusu zamansal ve yapısal zafiyetleri deterministik bir şekilde aşmak amacıyla, dar koridor kısıtları altında çalışan AGV hatları için kural ve öncelik tabanlı özgün bir melez CA* algoritması önerilmiştir. Geliştirilen model kapsamında araçların özel önceliklerini, hat üzerindeki fiziksel konumlarını, hedefe kalan mesafelerini ve toplam iş yüklerini içeren dört bileşenli bir sözlüksel öncelik vektörü ρ_i tanımlanmış ve yapay kilitlenmeleri engellemek adına dinamik bir gecikmeli çıkış (*Delayed Release*) mekanizması sisteme entegre edilmiştir. Ayrıca ana yol üzerinde duraklamayı tamamen yasaklayan bir kısıt kurgulanmış gereksiz manevraları önlemek için asimetrik bir maliyet fonksiyonu uygulanmış ve çoklu görev yönetimini optimize etmek adına En Kısa İş İlk (*SJF*) yaklaşımı araç içi görev sıralamasına dahil edilmiştir. Yapılan kapsamlı senaryo analizleri ve stres testleri sonucunda, anlık fiziksel güvenlik mesafesi ihlalleri dışında algoritmik veya yapısal herhangi bir kilitlenme ya da kafa kafaya çarpışma hatasıyla karşılaşmadığı ve sistemin başarıyla ölçeklendiği gözlemlenmiştir.

Çalışmanın geri kalanı belirli bir akışla düzenlenmiştir. [Bölüm 2](#)'de çok etmenli yol bulma problemleri ve işbirlikçi yol bulma yaklaşımlarına dair literatürde yer alan çalışmalar irdelenmiştir. [Bölüm 3](#)'te önerilen melez modelin matematiksel modeli, sistem kısıtları, soyut veri yapıları, asimetrik maliyet yapısı ve algoritmik karmaşıklığı açıklanmıştır. [Bölüm 4](#)'te zıt yön çatışması, darboğaz sınaması, stres testi ve özel öncelikli araç senaryoları üzerinden elde edilen deneysel bulgularla sistemin hesaplama performansı irdelenmiştir. [Bölüm 5](#)'te ise çalışmadan elde edilen temel sonuçlar tartışılmış ve gelecekte yapılabilecek geliştirmelere yönelik önerilere yer verilmiştir.

2 Literatür Taraması

2.1 Çoklu Ajanlı Yol Bulma (MAPF) Problemi

Endüstri 4.0'ın gelişimi ve otonom depo sistemlerinin yaygınlaşmasıyla birlikte, Otonom Yönlendirmeli Araçların (AGV) paylaşımlı alanlarda eş güdümlü hareketi literatürde büyük bir ivme kazanmıştır (Wu et al., 2023). Standart MAPF problemi (*Multi-agent Pathfinding*), etmenlerin birbirleriyle çarpışmadan, kilitlenme (*deadlock*) yaşamadan başlangıç noktalarından hedeflerine ulaşmalarını amaçlar. Ancak bu etmenlerin güzergahlarının, uzay ve zaman kısıtları altında optimal bir biçimde hesaplanması, problemi hesaplamasal olarak oldukça zorlayıcı (*genellikle NP-Zor*) bir yapıya dönüştürmektedir (Surynek, 2022).

2.2 MAPF Çözüm Yaklaşımları

Literatürdeki MAPF çözümleri genel olarak bütüncül (*Centralized/Optimal*) ve ayrık (*Decoupled/Suboptimal*) algoritmalar olmak üzere çeşitli dallara ayrılmaktadır. Çatışma Tabanlı Arama (*Conflict-Based Search - CBS*) (Sharon et al., 2015) veya MILP/SAT tabanlı derleme yöntemleri teorik olarak optimal çözümler sunsa da, sistemdeki araç sayısı arttıkça hesaplama maliyetleri gerçek zamanlı uygulanabilirliği imkansız hale getirebilmektedir (Surynek, 2022). Bu nedenle endüstriyel ve dinamik AGV navigasyonunda hız, ölçeklenebilirlik ve pratikliği bir araya getiren Öncelik Tabanlı (*Priority-Based*)¹ ve Kural Tabanlı (*Rule-Based*)² yaklaşımlar daha sık tercih edilmektedir (Wu et al., 2023). Öğrenme tabanlı yaklaşımlar literatürde yer bulmaya başlasa da, bu yöntemler henüz gerçek saha dağıtımlarından ve kesin kural doğrulamasından uzak kabul edilmektedir.

2.3 Öncelikli Planlama ve Cooperative A*

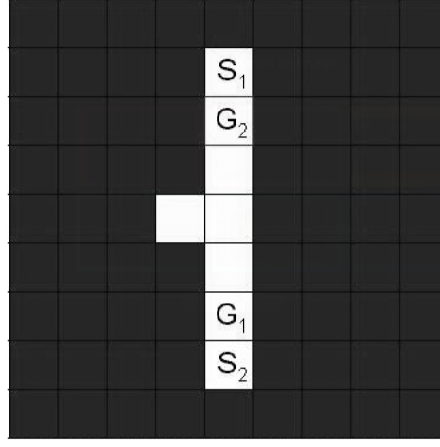
Öncelik tabanlı planlamanın temel taşlarından biri, David Silver tarafından literatüre kazandırılan İşbirlikçi Yol Bulma (*Cooperative Pathfinding*) ve buna bağlı Cooperative A* (CA*) algoritmasıdır (Silver, 2005). Ayrık bir çözüm sunan CA*, etmenlerin harita üzerindeki hareketlerini eş güdümlü hale getirmek için zaman boyutunu da içeren bir Uzay-Zaman Yer Ayırma Tablosu (*Space-Time Reservation Table*) kullanır. Bu mimaride, etmenlere belirli bir öncelik sırası atanır. Her etmen, uzay-zaman matrisinde kendi güzergahını bulurken, kendinden daha yüksek öncelikli etmenlerin belirli bir t -anında ayırttığı konum düğümlerinden kaçınır. Bu sayede dinamik engellerden kaçınma işlemi, üç boyutlu (x, y, t) bir yol arama problemine indirgenmiş olur.

2.4 Önerilen Değişiklik

Silver (2005)'teki esas CA* algoritması geniş ızgara (*grid*) tabanlı haritalarda başarılı sonuçlar verse de, dar koridorlar ve tek şeritli endüstriyel topolojilerde iki temel zafiyete sahiptir: (i) Eşit öncelikli etmenlerin çizelgeleme sırasının çözüme etkisi ve (ii) dar alanlarda kaçacak yer bulamayan etmenlerin oluşturduğu Şekil 1'deki koridor kilitlenmeleri (*corridor deadlocks*) (Silver (2005)). Literatürdeki derleme tabanlı veya bütüncül çözümler bu kilitlenmeleri aşabilse de hesaplama maliyetleri AGV sistemleri için çok yüksektir (Surynek (2022)).

¹(Ma et al., 2019)

²(Sajid et al., 2021) (Okumura et al., 2022) (Wang and Rubenstein, 2020)



Şekil 1: Bu basit problem CA* ile çözülememektedir. Bir etmen S1'den G1'e yol alması gerekirken diğeri S2'den G2'ye hareket etmeye çalışmaktadır. (Silver, 2005)

Bu çalışmada, tek şeritli yol yapısının getirdiği kısıtlar doğrultusunda klasik CA* algoritması üzerinde dört aşamalı, özgün bir melez (kural ve öncelik tabanlı) değişiklikler gerçekleştirilmiştir:

- **Çizelgeleme ve Gecikmeli Çıkış Mekanizması:** Klasik algoritmalarındaki rastgele veya statik sıralama yerine; araçların özel öncelik durumunu $(1 - v_i)$, yolun ilerisindeki araçların trafiği önce boşaltması amacıyla başlangıç konumunu $(-x_{\text{başlangıç},i})$, hedefe kalan mesafelerini (d_i) ve toplam görev yüklerini $(-W_i)$ kapsayan dört bileşenli bir sözlüksel öncelik vektörü $\rho_i = (1 - v_i, -x_{\text{başlangıç},i}, d_i, -W_i)$ (Bölüm 3.3.4) tanımlanmıştır. Ayrıca, uzay-zaman matrisinin yoğunluğundan kaynaklanan yapay kilitlenmeleri (*artificial deadlock*) engellemek adına, geçerli rota bulamayan araçların sisteme giriş anı $(t_{\text{başlangıç}})$ dinamik olarak ötelenmiştir (*Delayed Release*). Bu sayede sisteme araç enjeksiyonu optimize edilmiştir.
- **Yolda Bekleme Yasağı:** Olası fiziksel zıt yön çatışmalarını engellemek adına, araçların ana transit yol üzerinde durarak bekleme yapması (*wait action*) tamamen yasaklanmıştır. Tüm durma ve yol verme eylemleri, kapasite kısıtlı cepler (*C*) ve ara depolar (*G*) ile sınırlandırılmıştır (Bölüm 3.1.4).
- **Asimetrik Maliyet Fonksiyonu:** Algoritmanın araçları rotayı uzatan gereksiz çık-dön (*yo-yo*) manevralarından kaçındırması adına asimetrik bir maliyet yapısı kurgulanmıştır (Bölüm 3.3.2). Hareket eyleminin maliyeti 1.0 olarak belirlenirken, güvenli alanlarda (cep, depo, başlangıç) bekleme maliyeti 0.9 olarak atanmıştır. Bu ağırlıklandırma, araçların ana yolu işgal etmek yerine matematiksel olarak güvenli noktalarda duraklamasını teşvik etmektedir.
- **Araç İçi Görev Sıralaması (SJF):** Çoklu depo ziyareti gerçekleştirecek araçların kendi içindeki görev rotaları, En Kısa İş İlk (*Shortest Job First*) yaklaşımıyla optimize edilmiştir (Bölüm 3.3.3). Depolar başlangıç noktasına olan mutlak uzaklıklarına göre yakından uzağa sıralanarak, kısa mesafeli görevlerin hızla bitirilmesi, ana yolun erken boşaltılması ve uzay-zaman matrisindeki rezervasyon çakışmalarının minimize edilmesi sağlanmıştır.

Önerilen bu dinamik çizelgeleme, asimetrik maliyet yönetimi ve alana özgü kısıt tümleştirilmesi, MAPF literatüründeki dar koridor kilitlenmesi problemini bu projenin kısıtları dahilinde deterministik ve düşük bir hesaplama maliyetiyle çözüme kavuşturmuştur.

3.1.3 Amaç Fonksiyonu (Makespan Minimizasyonu)

Tüm araçların görevlerini tamamladığı toplam süreyi minimize etmek hedeflenmektedir:

$$\min \max_{i \in A} y_i \quad (1)$$

3.1.4 Sistem Kısıtları

Hareket Denklemi Araçlar her adımda en fazla bir düğüm ilerleyebilir veya bekleyebilir.

$$x_i(t+1) = x_i(t) + \Delta x, \quad \Delta x \in \{-10, 0, 10\} \quad (2)$$

Güvenlik Mesafesi Kısıtı İki araç arasındaki mesafe her zaman güvenli sınırlar içinde kalmalıdır. *Bu kural cepler ve depolar dışındaki ana yolda mutlak geçerlidir.*

$$\forall i \neq j, \forall t : |x_i(t) - x_j(t)| \geq d_{\min} = 20 \text{ m} \quad (3)$$

Kapasite Kısıtı Herhangi bir t -anında bir cepte en fazla 1 araç, bir ara depoda ise en fazla 3 araç bulunabilir.

Bir diğer ifadeyle $\forall t$ için, $x \in C$ konumundaki araç sayısı ≤ 1 . $x \in G$ araç sayısı ≤ 3 olmalıdır dolayısıyla:

Cepler için:

$$\sum_{i \in A} \mathbf{1}(x_i(t) = c) \leq 1, \quad \forall c \in C, \forall t \in T \quad (4)$$

Depolar için:

$$\sum_{i \in A} \mathbf{1}(x_i(t) = g) \leq 3, \quad \forall g \in G, \forall t \in T \quad (5)$$

biçiminde ifade edilebilir.

$\mathbf{1}(x_i(t) = c)$: Gösterme (*indicator*) fonksiyonudur. Eğer o an araç c -konumundaysa 1, değilse 0 değerini alır.

Görev Kısıtı Her araç, turunu tamamlamadan önce en az bir depoyu ziyaret etmelidir. Her araç turunu tamamladığında ($t = y_i$ anında), konumu $x_i(y_i) = 0$ olmalı ve $t < y_i$ için en az bir kez $x_i(t) \in G$ sağlanmalıdır.

$$\exists t < y_i \text{ öyle ki } x_i(t) \in G \text{ ve } x_i(y_i) = 0 \quad (6)$$

Bekleme Eylemi Kısıtı Bekleme eylemi yani herhangi bir aracın konumunu bir sonraki $t + 1$ adımında değiştirmemesi ($x_i(t + 1) = x_i(t)$), ancak bulunduğu konumun cepler (C), depolar (G) veya başlangıç (0) kümesinde olmasıyla mümkündür.

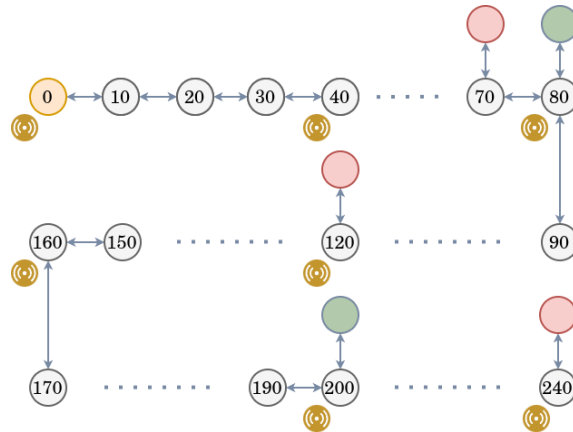
$$(x_i(t + 1) = x_i(t)) \implies x_i(t) \in C \cup G \cup \{0\} \quad (7)$$

3.2 Veri Yapıları ve Temsil

Algoritmanın karmaşık kısıtlar altında çözüm üretebilmesi için yol ağı, araçlar ve zaman boyutu belirli veri yapıları kullanılarak soyutlanmıştır. Bu temsiller, A* algoritmasının durum uzayını verimli bir şekilde taramasına olanak tanır.

3.2.1 Düğüm ve Çizge Yapısı

Sistemdeki her bir fiziksel nokta, $(konum_id, tür)$ şeklinde bir üçlü demet olarak temsil edilir. Burada $konum_id$ aracın yol üzerindeki metresini (0-240), $tür$ ise bulunduğu alanın fonksiyonunu (yol, cep, depo, başlangıç) belirtir. Bu ikili gösterim sayesinde, aynı koordinatta bulunan ancak farklı kapasite ve kurallara sahip olan *ana yol* ve *cep* düğümleri birbirinden ayrıştırılır. Çizge üzerindeki komşuluk ilişkileri, ana yol düğümlerinin birbirine doğrusal ($\pm 10m$) bağlanması ve işlevsel düğümlerin (cep/depo) sadece kendi konumlarındaki yol düğümüne bağlanmasıyla kurulur.



Şekil 3: Kullanılan Çizge Yapısı

Örneğin $(70, 'depo')$ düğümü sadece $(70, 'yol')$ düğümüne bağlı olurken $(70, 'yol')$ düğümü $(60, 'yol')$, $(70, 'depo')$ ve $(80, 'yol')$ düğümlerine bağlı olacaktır. Aynı durum cep düğümleri için de geçerlidir.

Düğümler arası bağlantıları (Kenar Kümesi - E) veya komşuluk fonksiyonunu ($N(v)$) tanımlayabiliriz. Bir (x, τ) düğümünün komşuluk fonksiyonu $N(x, \tau)$ şu şekilde formüle edilebilir:

Ana yol düğümlerinin komşuları (hem yolda ileri/geri, hem de varsa bulunduğu konumdaki cep/de-poya giriş):

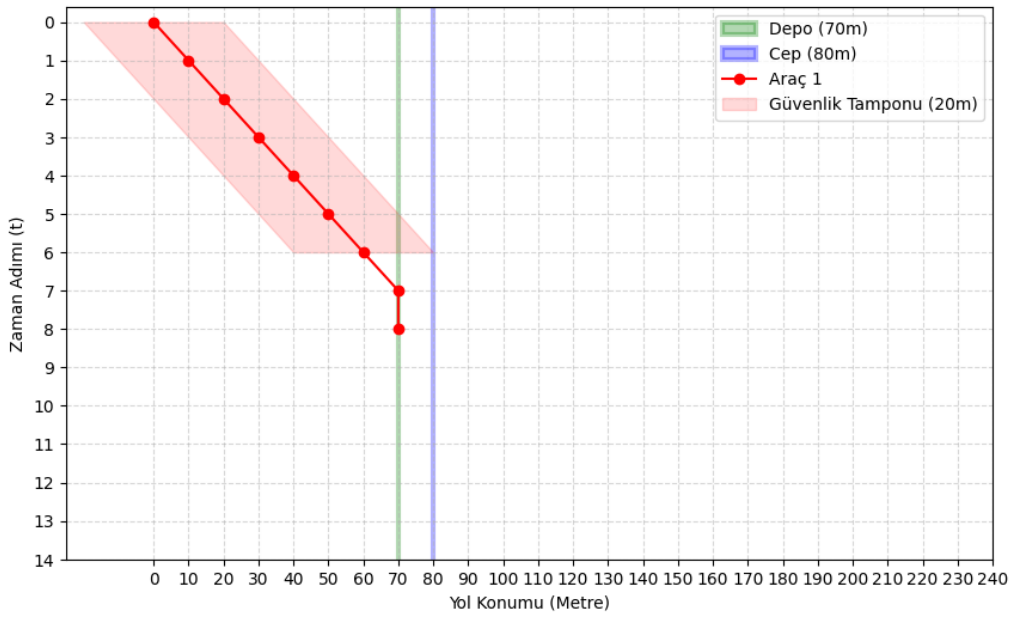
$$N(x, 'yol') = \{(x \pm 10, 'yol')\} \cup \{(x, \tau) \mid (x \in C \wedge \tau = 'cep') \vee (x \in G \wedge \tau = 'depo')\} \quad (8)$$

Cep veya depo düğümlerinin komşuları (kendi içinde bekleme veya yola çıkma):

$$N(x, \tau) = \{(x, \tau), (x, 'yol')\} \quad \tau \in \{'cep', 'depo'\} \quad (9)$$

3.2.2 Durum Uzayı (State Space) Tanımı

A* arama algoritması, sistemi statik bir harita yerine Zaman-Genişletilmiş Ağ (*Time-Expanded Network*) üzerinde değerlendirir. Arama uzayındaki her bir durum, (düğüm, zaman) sıralı ikilisiyle tanımlanır. Bu yapı, aracın bir düğümde *bekleme* eylemi yapmasını, zaman ($t \rightarrow t + 1$) ilerlerken düğüm bilgisinin sabit kalması biçiminde modellemeyi sağlar. Arama, aracın başlangıç noktasındaki durumundan başlar ve hedef düğümün herhangi bir t -anındaki durumuna ulaşıldığında parça (*segment*) bazlı olarak tamamlanır.



Şekil 4: Güvenlik Tamponu ve Uzay-Zaman Grafiği

A* algoritmasının arama uzayı (S), fiziksel düğümler kümesi (V) ile zaman adımları kümesinin (T) Kartezyen çarpımıdır. Düğüm kümesi V ise konumlar (X) ve tür (Γ) uzayının çarpımıdır.

$$S = V \times T = \{(x, \tau, t) \mid x \in X, \tau \in \Gamma, t \in T\} \quad (10)$$

Burada $X = \{0, 10, \dots, 240\}$ ve $\Gamma = \{yol, cep, depo, başlangıç\}$ olarak düşünülebilir.

Şekil 4'deki senaryoda araç $x = 0$ ve $t = 0$ konumundan başlayarak sabit hızla $x = 70$ konumundaki depoya ulaşmaktadır. Depoya girdiği andan itibaren güvenlik tamponu tahsisi kalkmaktadır.

3.2.3 Merkezi Rezervasyon Tablosu ve Güvenlik Tamponu

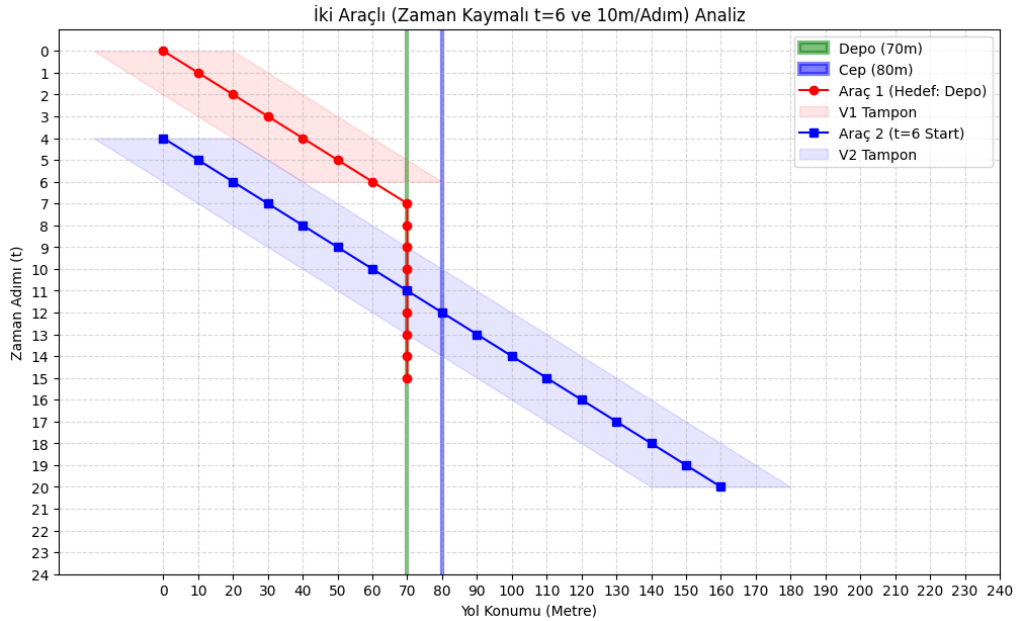
Araçlar arası çarpışmaları ve güvenlik mesafesi ihlallerini önlemek için merkezi bir rezervasyon veri yapısı kullanılmıştır. Bu yapı, (konum, tür, zaman) anahtarlarını karşılık gelen doluluk bilgisiyle eşleştiren bir sözlük (dictionary/hash map) olarak gerçekleştirilmiştir. Bir araç için güzergah kesinleştiğinde, güzergahtaki her bir adım tabloya işlenir. Güvenlik mesafesi ($d_{min} = 20$) kuralını sağlamak adına, bir araç t -anında x -konumunda bulunuyorsa, tablodaki $x - 20$, $x - 10$, x , $x + 10$, $x + 20$ noktaları o zaman dilimi için rezerve edilerek diğer araçların bu bölgeye girmesi engellenir.

i -aracının t -anında oluşturduğu güvenlik tamponunu (rezerve edilen konumlar kümesi) $B_i(t)$ olarak tanımlarsak:

$$B_i(t) = \{x_i(t) + \delta \mid \delta \in \{-20, -10, 0, 10, 20\}\} \quad (11)$$

Diğer araçların ($j \neq i$) çarpışma kısıtını sağlaması için rezervasyon tablosu gereği bu kümenin elemanı olmaması gerekir.

$$x_j(t) \notin B_i(t), \quad \forall j \neq i \quad (12)$$



Şekil 5: Güvenlik Tamponu ve Uzak-Zaman Grafiği

Şekil 5'deki senaryoda ilk araç (0,0) konumundaki hareketi sonrasında (70, t) konumundaki depoya ulaşmaktadır. İkinci araç ise güvenlik mesafesini koruyarak aracı takip ederek ilerelemektedir. Dikkat edilirse ilk araç depoya girdiğine güvenlik tamponu oluşmamaktadır. Öte yandan ikinci araç ise deponun konumundan geçerken güvenlik tamponu tahsis etmeye devam etmektedir. Bunun nedeni aracın depoya girmemesidir.

3.2.4 Araç ve Görev Yönetimi

Araçlar, kimlik bilgileri, öncelik durumları ve sıralı görev dizelgelerini içeren nesneler olarak tanımlanmıştır. Görevlerin parçalı yapısı (örneğin: $0 \rightarrow D70 \rightarrow 0 \rightarrow D240 \rightarrow 0$), toplam yolculuğun

segmentlere bölünmesini ve her segmentin bir önceki bitiş zamanını başlangıç kabul ederek ardışık olarak planlanmasını sağlar.

Bir i -aracının tüm görev rotasını (M_i), başlangıç noktasından (0) başlayıp aralarda G kümesindeki depolara uğradığı ve sonunda yine başlangıca döndüğü sıralı n -demeti (ya da dizi) olarak ifade edebiliriz:

$$M_i = \langle 0, g_{i,1}, 0, g_{i,2}, 0, \dots, g_{i,k}, 0 \rangle \quad (13)$$

$$\text{öyle ki } g_{i,m} \in G, \quad \forall m \in \{1, 2, \dots, k\} \quad (14)$$

3.3 Algoritma ve Çizelgeleme Yöntemi

Problem, İşbirlikçi A* (*Cooperative A**) algoritması kullanılarak merkezi bir rezervasyon tablosu üzerinden çözülmektedir. Bu strateji, araçların birbirlerinin gelecek rotalarını bilerek hareket etmesini sağlar.

3.3.1 Uzak-Zaman Rezervasyonu ve A* Taraması

Her araç için rota planlaması yapılırken, 3 boyutlu bir durum uzayı (Konum $\times$ Tip $\times$ Zaman) taranır. Algoritma bir rota bulduğunda, bu rotadaki tüm konumları ve güvenlik tamponlarını (20m mesafe kuralı gereği komşu düğümleri) merkezi rezervasyon tablosuna işler. Böylece sonraki araçlar, planlarını yaparken önceden rezerve edilmiş bu alanlardan kaçınır.

3.3.2 Maliyet Fonksiyonu

Algoritmanın araçları gereksiz manevralardan (*yo-yo hareketi*) kaçındırması için asimetrik bir maliyet yapısı uygulanmıştır:

$$c(s_t, s_{t+1}) = \begin{cases} 1.0, & \text{Eğer araç hareket ediyorsa} \\ 0.9, & \text{Eğer araç güvenli noktada (cep/depo/başlangıç) bekliyorsa} \end{cases} \quad (15)$$

Bu düşük bekleme maliyeti, aracın yolu meşgul etmek yerine güvenli bir noktada durmasını teşvik eder. Böylece algoritma, hedefe yaklaşamayıp yolu uzatan çık-dön (*yo-yo*) manevralarını $1.0 + 1.0 = 2.0$ maliyetle değerlendirirken, güvenli bir cepte/başlangıçta 2 birim beklemeyi $0.9 + 0.9 = 1.8$ maliyetle daha kârlı bulur ve gereksiz güzergahlar önlenir.

Maliyet fonksiyonu $c(s_t, s_{t+1})$, A* algoritmasının değerlendirme fonksiyonu olan $f(s) = g(s) + h(s)$ içerisinde başlangıçtan o anki duruma kadar olan kesin maliyeti ($g(s)$) hesaplamak için kullanılır:

$$g(s_{t+1}) = g(s_t) + c(s_t, s_{t+1}) \quad (16)$$

Ayrıca algoritmanın hedef düğüme kalan tahmini mesafeyi hesapladığı sezgisel fonksiyon ($h(s)$), 1 boyutlu ağ üzerinde adım sayısı cinsinden *Manhattan mesafesi* olarak tanımlanmıştır. Adım büyüklüğü $\Delta x = 10$ m olmak üzere:

$$h(s_t) = \frac{|x(s_t) - x_{hedef}|}{\Delta x} \quad (17)$$

3.3.3 Araç İçi Görev Sıralaması (*Local Task Sorting*)

Algoritmanın genel performansını ve kilitlenme ihtimalini doğrudan etkileyen faktörlerden biri, her bir aracın kendi içindeki görevleri (ziyaret edeceği depoları) hangi sırayla gerçekleştirdiğidir. Bu çalışmada, araç içi görev sıralaması için En Kısa İş İlk (*Shortest Job First - SJF*) yaklaşımı kullanılmıştır. Bir araç birden fazla depoyu ziyaret edecekse, rotası başlangıç noktasına en yakın olan hedeften başlayacak şekilde sıralanır. Bu yöntemin temel amacı, araçların kısa mesafeli görevlerini hızla tamamlayıp ana yolu erken terk etmesini sağlamaktır. Ana yolun hızlı bir şekilde boşalması, diğer araçların hareket alanını genişleterek uzay-zaman tablosundaki rezervasyon çakışmalarını en aza indirir ve sistemin toplam tamamlanma süresini (*makespan*) optimize eder.

Bir i -aracına atanmış sırasız hedef depolar kümesi $D_i \subseteq G$ olmak üzere, bu kümedeki elemanlar başlangıç noktasına (0. konum) olan mutlak uzaklıklarına göre küçükten büyüğe sıralanır. Bu işlem sonucunda $\langle g_{i,1}, g_{i,2}, \dots, g_{i,k} \rangle$ sıralı dizisi elde edilir ve sıralama kısıtı aşağıdaki eşitsizliği sağlar:

$$|g_{i,1}| \leq |g_{i,2}| \leq \dots \leq |g_{i,k}| \quad (18)$$

Burada her bir $g_{i,m} \in D_i$, aracın sırayla gideceği depo konumudur. Otonom araçların her yük alımından sonra başlangıç noktasına dönme kuralı gereği, bu sıralı hedefler arasına 0. konum eklenerek aracın nihai parçalı rota dizisi (M_i) kurgulanır:

$$M_i = \langle 0, g_{i,1}, 0, g_{i,2}, 0, \dots, g_{i,k}, 0 \rangle \quad (19)$$

3.3.4 Önceliklendirme ve Kilitlenme Çözümü

Sistemdeki araçlar, kilitlenmeleri (*deadlock*) önlemek için belirli bir hiyerarşiye göre sırayla planlanırlar:

1. **Özel Öncelik Durumu:** Öncelikli araçlar her zaman ilk sırada planlanırlar.
2. **Fiziksel Konum Önceliği:** Tek şeritli yol hatlarında sollama imkanı olmadığından, yapay kilitlenmeleri önlemek adına hat üzerinde (0. metreden) daha ileride konumlanan araçlara öncelik verilerek öncelikle onların yolu boşaltılması sağlanır.
3. **Hedefe Yakınlık:** Mevcut hedefine daha yakın olan araçlara öncelik verilerek yolun öncelikle onlar tarafından boşaltılması sağlanır.
4. **Gecikmeli Çıkış (*Deadlock Resolver*):** Eğer bir araç için geçerli bir rota bulunamazsa, bu durum sistemin o anki yoğunluğu nedeniyle kilitlenmeye gittiğini gösterir. Çözüm olarak aracın yola çıkış zamanı ($t_{başlangıç}$) belirli bir adım kadar ötelenerek başlangıç noktasında bek-

letilir ve algoritma tekrar çalıştırılır. Bu mekanizma, yolda güvenli boşluklar oluşana kadar araçların sisteme girişini düzenleyerek yapay kilitlenmeleri (*artificial deadlock*) engeller.

Her i -aracı için çizelgeleme önceliği, üç bileşenli bir öncelik vektörü (*tuple*) ρ_i ile tanımlanmıştır. Algoritma bu vektörleri küçükten büyüğe değerlendirir.

Öncelik vektörünün bileşenleri şunlardır:

1. $v_i \in \{0, 1\}$: Aracın özel öncelik durumu (öncelikli ise 1, değilse 0). Öncelikli araçların öne geçmesi için bu değer ters çevrilerek $(1 - v_i)$ olarak kullanılır.
2. $-x_{\text{başlangıç},i}$: Aracın yola çıktığı orijinal başlangıç noktası. Hat üzerinde daha ileride olan aracın küçük bir değer olarak öne geçmesi için bu değer negatif olarak eklenir.
3. d_i : Aracın mevcut hedefine olan uzaklığı ($d_i = |x_{\text{hedef}} - x_i(t)|$).
4. W_i : Aracın tüm görevleri boyunca kat edeceği toplam mesafe. Daha çok işi olan aracın öne geçmesi için bu değer negatif ($-W_i$) olarak vektöre eklenir.

Buna göre i -aracının öncelik vektörü ρ_i şu şekilde ifade edilir:

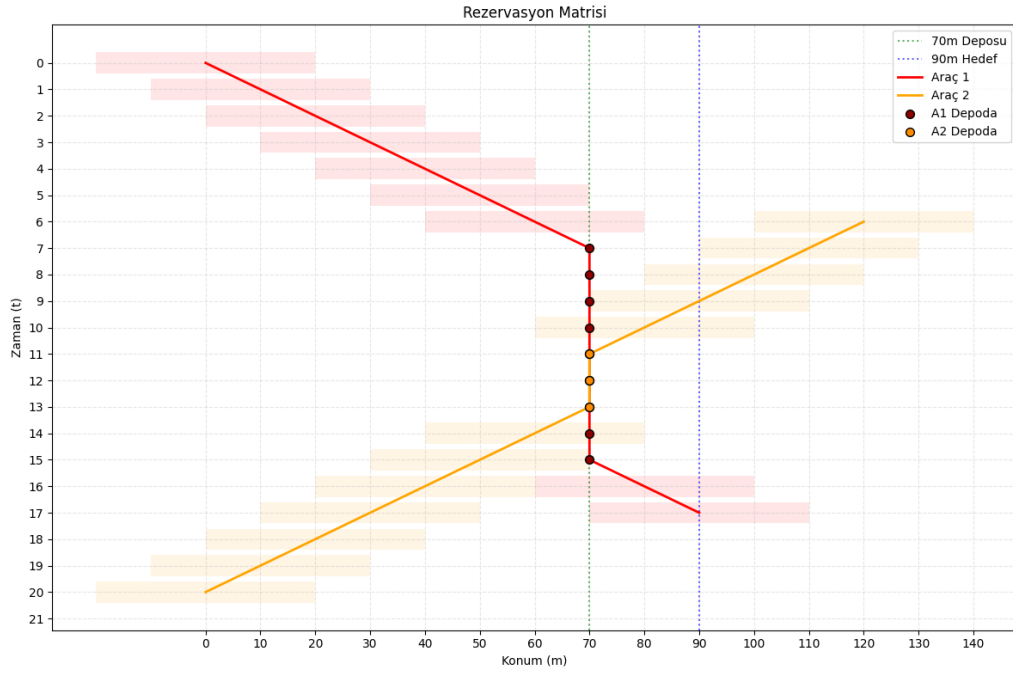
$$\rho_i = (1 - v_i, -x_{\text{başlangıç},i}, d_i, -W_i) \quad (20)$$

Yalnızca hedefe olan uzaklığın (d_i) baz alınması, araçların farklı istasyonlara yöneldiği senaryolarda **yapay kilitlenmelere** yol açmaktadır. Örneğin; 20. ve 30. metrelerde konumlanan iki araçtan, 20. metredeki hedefi Depo 70, 30. metredeki hedefi ise Depo 120 olsun. Sadece hedefe yakınlık kriteri uygulandığında, 20. metredeki araç hedefine daha yakın olduğu için ($50 < 90$) ilk sırada çizelgelenmeye çalışılacaktır. Ancak tek şeritli hat üzerinde önünde bulunan 30. metredeki araç henüz hareket etmediği için $d_{\min}$ güvenlik tamponu anında ihlal edilecek ve kilitlenme oluşacaktır. Vektöre eklenen $-x_{\text{başlangıç},i}$ bileşeni sayesinde, hat üzerinde fiziksel olarak daha önde olan (30. metredeki) araç ilk önce çizelgelenir; böylece güvenlik tamponunu arkasından gelen araç için temizleyerek yapay kilitlenmeyi engeller.

Özel öncelikli araçlar daima başlangıç noktasında (0. metre) konumlanır ve tek bir görev (belirli bir depoya gidip dönme) üstlenirler. Algoritma bu araçları mutlak ilk sıraya alarak çizelgeler. Bu sınırlamayla birlikte öncelikli araçların sistem henüz boşken tahsis edilmemiş bir uzay-zaman tablosunda ilerlemesini sağlar. Böylece diğer araçların yaratabileceği yoğunluğa ve gecikmelere takılmadan görevlerini matematiksel olarak mümkün olan en kısa sürede (*optimal makespan*) tamamlamaları garantilenir.

İki araç karşılaştırıldığında sözlüksel sıralama (*lexicographical order*) uygulanır. Eğer $\rho_i < \rho_j$ ise, i -aracı j -aracından önce çizelgelenir. Bu yapı sayesinde özel öncelikli araçlara mutlak üstünlük verilirken, eşitlik durumlarında sırasıyla hedefe yakınlık ve toplam iş yükü devreye girer.

Şekil 6'de daha önce çizelgelenmiş A_2 aracı ve yol vermek için depoya giren A_1 aracı bulunmaktadır. Bu örnekte A_2 aracı depoya uğradıktan sonra depodan ayrılmakta ardından A_1 aracı diğer aracın güvenlik tamponundan çıktıktan sonra depodan dışarı çıkmaktadır.



Şekil 6: Yol Verme Senaryosu

3.3.5 Algoritma Karmaşıklığı

İşbirlikçi A* algoritmasının zaman karmaşıklığı, fiziksel yol düğümlerinin (V) ve maksimum zaman adımının (T_{max}) oluşturduğu arama uzayına bağlıdır. Tek bir araç için en kötü durum (worst-case) zaman karmaşıklığı $O(|V| \times T_{max} \cdot \log(|V| \times T_{max}))$ olarak ifade edilebilir. Algoritmanın çalışma süresini asıl optimize eden unsur, merkezi rezervasyon tablosunun bir Hash Map (sözlük) veri yapısında kurgulanmasıdır. Bu sayede algoritmanın uzay-zaman tablosundaki çakışma ve güvenlik kontrolleri $O(1)$ sabit sürede yapılarak, dinamik araç sayısının yarattığı işlem yükü minimize edilmiştir.

4 Bulgular ve Senaryo Analizi

Senaryolardaki çizge yapısı Bölüm 3.1.1’de tanımlandığı gibidir. Analizlerde aksi belirtilmedikçe buradaki çizge yapılandırması kullanılacaktır.

4.1 Zıt Yön Çatışması

Bu senaryoda yazılan programın dinamik başlangıç özelliğinden yararlanarak araçların olası zıt yönde karşı karşıya gelme durumunda birbirlerine nasıl yol verdikleri analiz edilecektir.

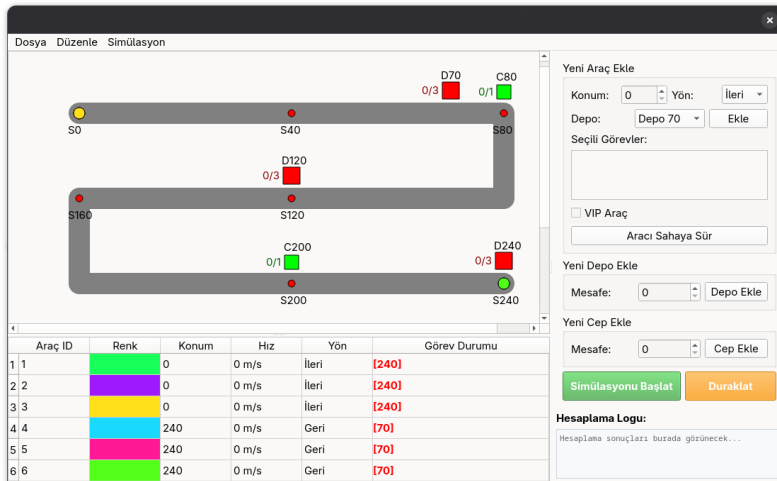
4.1.1 Yapılandırma

Araçlar, görevleri ve yönleri aşağıdaki gibi yapılandırılmıştır. Araçların başlangıcı başlık 3.3.4’de verilen çizelgeleme algoritmasına göre belirlenmektedir.

Araçlar $t = 0$ anında aşağıdaki konumlarda başlamaktadırlar. $A = \{1, 2, 3, 4, 5, 6\}$ araçları için $X_{t=0}$ -konum vektörü olmak üzere:

$$X_{t=0} = \{0, 0, 0, 240, 240, 240\} \quad (21)$$

- Görevler = $\{\{240\}, \{240\}, \{240\}, \{70\}, \{70\}, \{70\}\}$
- Yönler = $\{1, 1, 1, -1, -1, -1\}$
- $G = \{70, 120, 240\}$
- $C = \{80, 200\}$

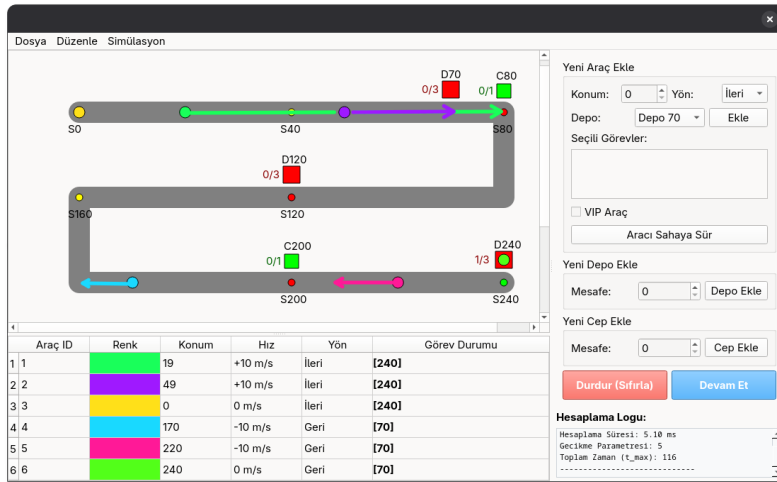


Şekil 7: Zıt Yön Çatışması: Başlangıç Durumu

4.1.2 Analiz

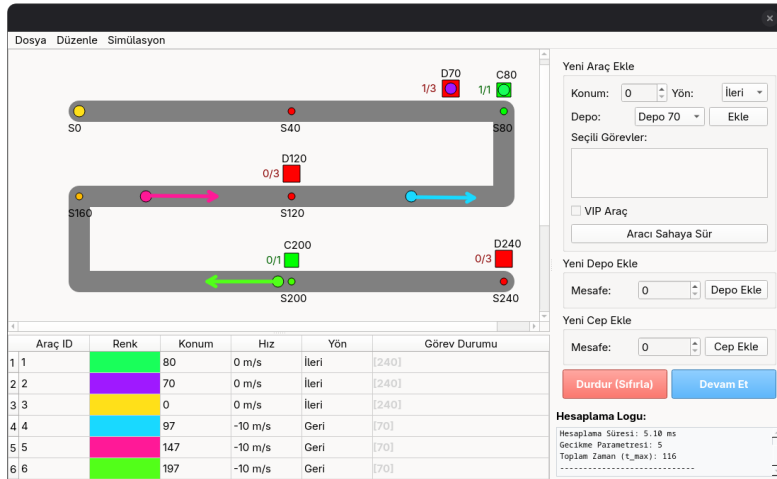
Başlangıç durumunda; (Şekil 7) D240’da üç araç sola doğru yönde başlamaktadır, başlangıç konumunda ($x = 0$) ise üç araç sağ yönde başlamaktadır. Çizelgeleme algoritması gereği (Bölüm 3.3.4)

önce D240'daki (A4, A5, A6) araçları önce çizelgelenmektedir. A5 ve A6 depoda beklerken A4 aracı D70'e doğru yola çıkmaktadır. Ardından güvenlik mesafesi korunarak A5 ve A6 araçları da D70 görevi için sırayla yola çıkıyorlar. Bu esnada başlangıç konumundaki A1, A2 ve A3 araçlarından A1 ve A2 yol vermek için D70 ve C80'de beklemektedirler (Şekil 9).



Şekil 8: İlk araçların çıkışı

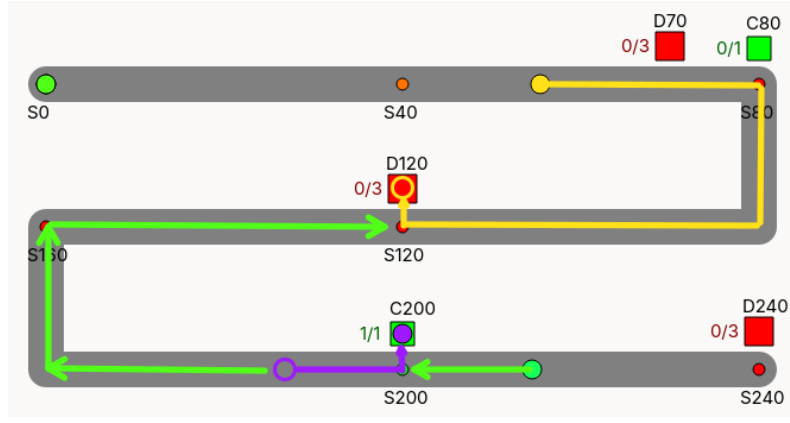
A4, A5, A6 araçları D70 görevini tamamlayıp başlangıca döndükten sonra görevleri bitiyor. Ardından D70 ve C80'de bekleyen A1 ve A2 araçları sırayla D240 görevi için yola çıkmaktadırlar. Bu iki araçtan sonra A3 aracı en son yola çıkıp D120'de bu iki araca yol vermek için beklemektedir. Çizelgelemeye göre en başta olan A1 aracı yükünü alırken A2 aracı C200 cebinde bekleyip yol vermektedir. En son A3 aracı diğer araçlara yol verdikten sonra yükünü D240'dan alıp başlangıca döner (Şekil 10).



Şekil 9: A1 ve A2 araçlarının beklemesi

Potansiyel Çarpışma Noktası Kontrol mekanizması olmasaydı, zıt yönlü hareket eden A_1 ve A_4 araçları $t = 4$ anında $x = 120$ m koordinatında kilitlenme (*deadlock*) veya kafa kafaya çarpışma yaşayacaktı.

Kilitlenme Analizi A_1 aracı C_{80} cebine çekilerek bekletilmiş, böylece kilitlenme olasılığı matematiksel kısıtlara uygun olarak sıfıra indirilmiştir.



Şekil 10: A2 ve A3 aracının A1 aracına yol vermesi

4.1.3 Sonuç

Yapılan analiz sonucunda çizelgelemeye uygun olarak hareketlerine başlayan araçlar cepler ve depoları kullanarak birbirlerine yol vererek sırayla görevlerini tamamladıkları görülmüştür. Algoritmanın hesaplama süresi ve toplam zaman adımı (*makespan*) Tablo 1’de gösterilmiştir.

Tablo 1: Sonuçlar

Hesaplama Süresi (ms)	5.10
Toplam Zaman Adımı (t_{max})	116

4.2 Darboğaz Sınaması

Bu senaryoda araçların hepsinin tek bir depoya yoğunlaştığı durumda düzeneğin nasıl davrandığı incelenecektir. Amaca uygun olarak 10 tane araç başlangıç konumunda ve hepsinin görevi sadece D70 olacak biçimde yapılandırma yapılmıştır. Ardından sistem optimizasyonu üzerine etkisini gözlemlemek için araçların kullanabileceği bir cep sisteme eklenerek aynı senaryo tekrarlanmıştır.

4.2.1 Yapılandırma

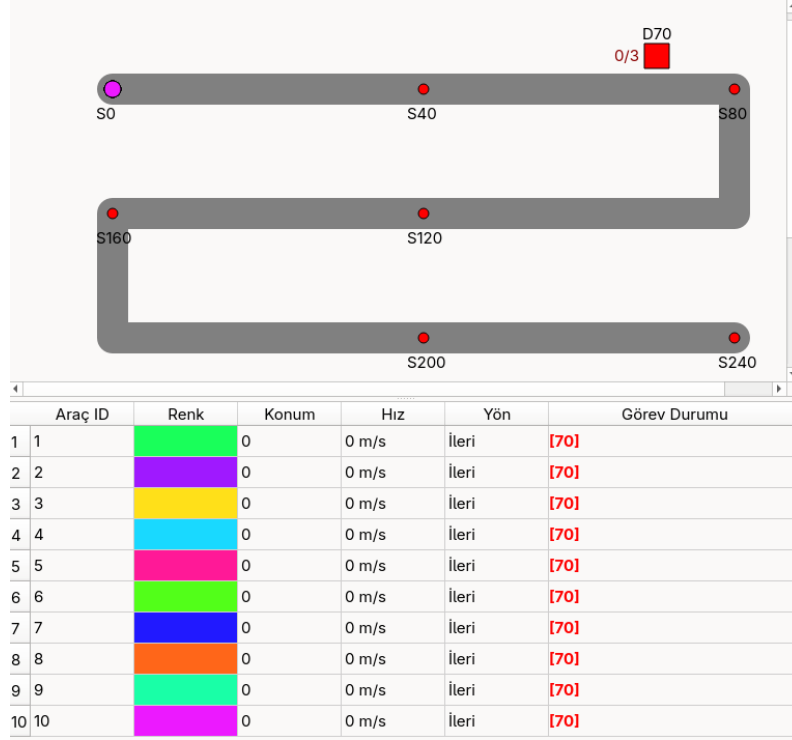
Araçlar $t = 0$ anında aşağıdaki konumlarda başlamaktadırlar. $A = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ araçları için $X_{t=0}$ -konum vektörü olmak üzere:

$$X_{t=0} = \{0, 0, \dots, 0\} \quad (22)$$

- Görevler = $\{\{70\}, \dots, \{70\}\}$
- Yönler = $\{1, \dots, 1\}$
- $G = \{70\}$
- $C = \emptyset$

4.2.2 Analiz

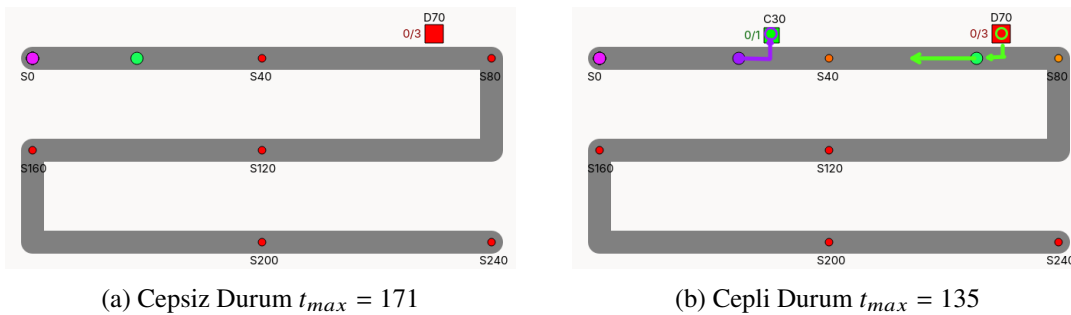
Araçlar çizelgeleme mantığına uygun olarak kimlik numaralarına göre sırayla başlangıçtan hareket edip D70 deposuna gitmektedirler (Şekil 11). Bir araç yoldayken başka bir araç depoya hareket etmemektedir (Şekil 12a). Arada bir cep olmadığından ve araçlar depoda yeteri kadar süre beklemediğinden darboğaz oluşmaktadır.



Şekil 11: Darboğaz Sınaması: Başlangıç Durumu

Bu durumu çözmek için araya bir cep eklenebilir (Şekil 12b). Cepler kümesi $C = \{30\}$ olacak biçimde güncellenerek Şekil 12b'deki yol yapılandırması elde edilmiştir. Cep eklendiği durumda iki araç peş peşe (güvenlik mesafesini koruyarak) çıkacaklar ve öndeki araç depodan dönüp aradaki cebi geçtikten sonra diğer cepteki araç depoya gidecektir. Cep boşaldıktan sonra ise üçüncü araç cebe girecektir. Bu durumu peş peşe çizelgelenen her araç için tekrarlanır.

Ceplerin eklenmesi durumunda araçlar hedeflerine daha yakın bir mesafeden başlayacaktır. Beklendiği üzere toplam zaman adımı (t_{max}) 171'den 135'e düşmüştür.



Şekil 12: Cepli ve Cepsiz Durumlarda Hareket Yönleri

4.2.3 Sonuç

Yapılan analiz sonucu açıkça görüldüğü üzere algoritma 10 aracın darboğaz oluşturduğu senaryoda tüm araçları başarıyla çizelgelemiş ve çarpışma durumu olmadan her aracın görevini tamamladığı gözlemlenmiştir. Darboğaz senaryosu depo ile başlangıç arasına bir cep eklenerek tekrarlandığında ise toplam zaman adımının azaldığı, hesaplama süresinin düştüğü ve araçların cepleri etkin biçimde kullanarak sistem optimizasyonunu artırdığı gözlemlenmiştir (Tablo 2).

Tablo 2: Sonuçlar

Senaryo	Hesaplama Süresi [ms]	Toplam Zaman Adımı (t_{max})
Cepsiz	6.58	171
Cepli	4.43	135

4.3 Stres Sınaması

Bu senaryoda sisteme görevleri değişen ve keyfi olarak atanmış, başlangıç noktasında konumlanan 21 araç verilmiştir (Şekil 13). Sistemin fazla araç durumunda davranışı incelencektir. Senaryo, ceplerin ve depoların kullanımı sonucu zaman adımı değişimini ölçmek adına 21 tane sadece D240 görevine sahip araç ile tekrarlanmıştır.

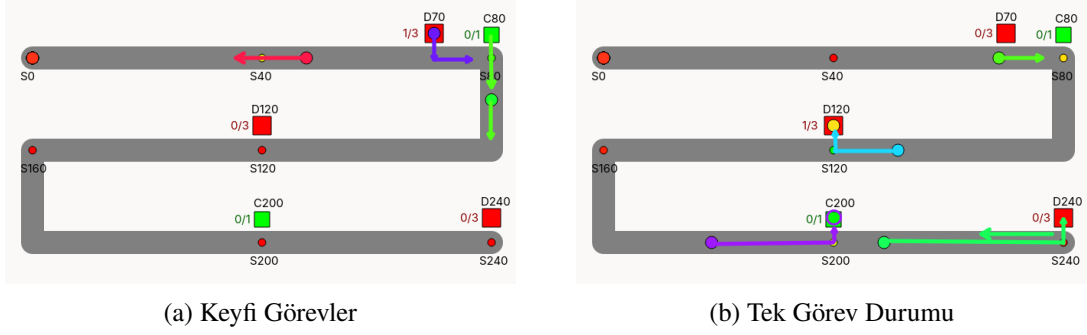
	Araç ID	Renk	Konum	Hız	Yön	Görev Durumu
1	1		0	0 m/s	İleri	[240]
2	2		0	0 m/s	İleri	[240]
3	3		0	0 m/s	İleri	[240]
4	4		0	0 m/s	İleri	[120]
5	5		0	0 m/s	İleri	[120]
6	6		0	0 m/s	İleri	[120]
7	7		0	0 m/s	İleri	[70]
8	8		0	0 m/s	İleri	[70]
9	9		0	0 m/s	İleri	[70]
10	10		0	0 m/s	İleri	[240]
11	11		0	0 m/s	İleri	[120]
12	12		0	0 m/s	İleri	[70] [120] [240]
13	13		0	0 m/s	İleri	[120] [240]
14	14		0	0 m/s	İleri	[120] [240]
15	15		0	0 m/s	İleri	[120] [240]
16	16		0	0 m/s	İleri	[70] [120]
17	17		0	0 m/s	İleri	[70] [120]
18	18		0	0 m/s	İleri	[70] [120]
19	19		0	0 m/s	İleri	[70] [120]
20	20		0	0 m/s	İleri	[120]
21	21		0	0 m/s	İleri	[120]

Şekil 13: 21 Araçlık Görev Tablosu

4.3.1 Analiz

$t_{max} = 526$ adımda ve 50.30 ms hesaplama süresiyle tüm araçlar çarpışma ve kilitlenme durumu olmadan görevlerini tamamlamıştır. Araçların görevleri boyunca cepleri ve depoları birbirlerine yol vermek için kullandıkları görülmüştür (Şekil 14). Araçların ceplerde veya depolarda beklerken gereksiz girme çıkma durumları yerine daima bekledikleri gözlemlenmiştir.

Araçların sadece D240 görevine sahip olduğu senaryodaysa (Şekil 14b) araçlar peş peşe başlangıçtan çıkmış ve sırayla uygun depo ve ceplerde önlerindeki aracın dönüşüne yol vermek için beklemeye geçmişlerdir.



Şekil 14: Görev Tablosu ve Sistemin Anlık Görüntüsü

4.3.2 Sonuçlar

21 araçlık senaryodaki hesaplamalar sistemde herhangi bir takılmaya neden olmadan kısa bir sürede tamamlanmıştır. Peş peşe birçok aracın Bölüm 4.2’de olduğu gibi aynı depoya gitmelerinden ötürü oluşan darboğaz gibi nedenlerden dolayı zaman adımının epey fazla olduğu gözlemlenmiştir.

Tek görevin olduğu senaryo ile kıyaslandığında (Tablo 3) hesaplama süresinin neredeyse 3 katına çıktığı toplam zaman adımının da 54-adım kadar düştüğü gözlemlendi. Zaman araçların başlangıçta zaman kaybetmek yerine depoda beklemesi sonucu azalsa da her t-anında yolda birçok aracın sürekli karşılaşmasından ötürü hesaplama süresi artmıştır.

Tablo 3: Sonuçlar

Görevler	Hesaplama Süresi [ms]	Toplam Zaman Adımı (t_{max})
Değişen	50.3	526
Tek	146.32	472

4.4 Özel Öncelikli Araç

Bu bölümde özel öncelikli olarak belirlenen araçlara diğer araçların yol vermesi incelenecektir.

4.4.1 Yapılandırma

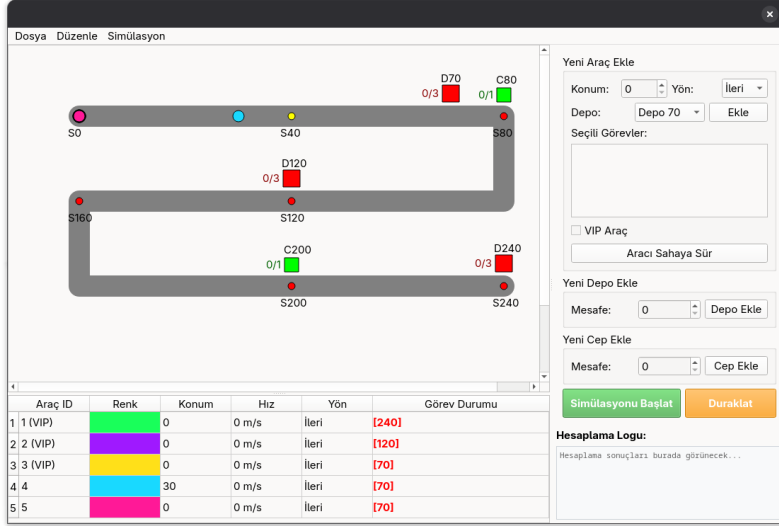
Araçlar $t = 0$ anında aşağıdaki konumlarda başlamaktadırlar. $A = \{1, 2, 3, 4, 5\}$ araçları için $X_{t=0}$ -konum vektörü olmak üzere:

$$X_{t=0} = \{0, 0, 0, 30, 0\} \quad (23)$$

A1, A2 ve A3 araçları özel öncelikli (VIP) araçlar olmaktadır. Her araç daima bu araçlara yol önceliği tanımaktadır.

- Görevler = $\{\{240\}, \{120\}, \{70\}, \{70\}, \{70\}\}$

- $Y\ddot{o}nler = \{1, 1, 1, 1, 1\}$
- $G = \{70, 120, 240\}$
- $C = \{80, 200\}$

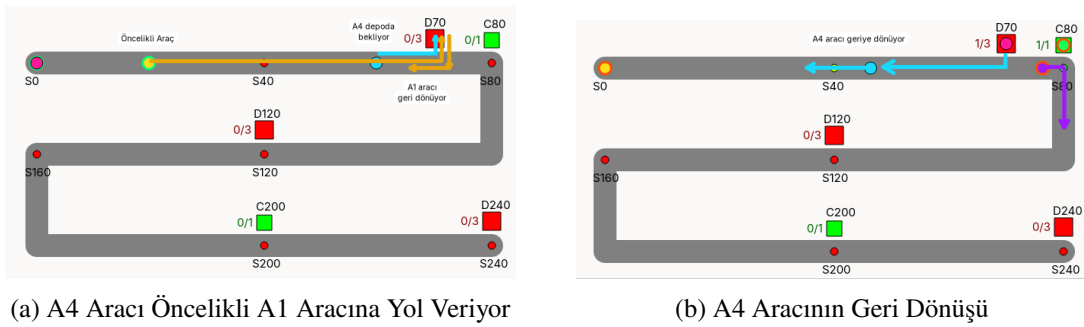


Şekil 15: Öncelikli Araçlar: Başlangıç Durumu

A4 aracı ($x = 30, t = 0$) konumundan başlatılarak bu aracın öncelikli araçlardan önde yer alması ve öncelikli araçlara yol verme davranışının incelenmesi amaçlanmıştır. (Şekil 15)

4.4.2 Analiz

Başlangıçta A4 aracı D70 deposuna girdikten sonra A1, A2 ve A3 araçları $x = 70$ konumunu geçip geri dönüş yolunu boşaltana kadar depoda beklemeye devam etmektedir. Başka bir ifadeyle A4 aracı öncelikli araçlar yolu boşaltana kadar yol hakkın kendisinde olmasına rağmen depoda beklemeye devam etmektedir.



Şekil 16: Öncelikli Araçlara Yol Verilmesi

4.5 Dinamik Başlangıç

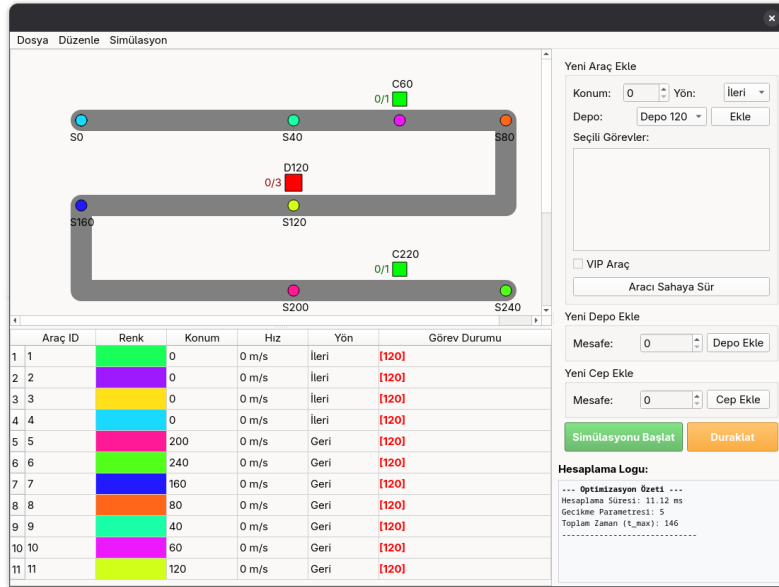
Bu senaryoda başlangıç dışında yolun ortasında başlayan araçlar ve tek bir depo kullanılarak algoritmanın dinamik başlangıçlarda kilitlenmeyi nasıl çözdüğü incelenecektir.

4.5.1 Yapılandırma

Araçlar $t = 0$ anında aşağıdaki konumlarda başlamaktadırlar. $A = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}$ araçları için $X_{t=0}$ -konum vektörü olmak üzere:

$$X_{t=0} = \{0, 0, 0, 0, 200, 240, 160, 80, 40, 60, 120\} \quad (24)$$

- Görevler = $\{\{120\}, \dots\}$
- Yönler = $\{1, 1, 1, 1, -1, \dots -1\}$
- $G = \{120\}$
- $C = \{60, 220\}$

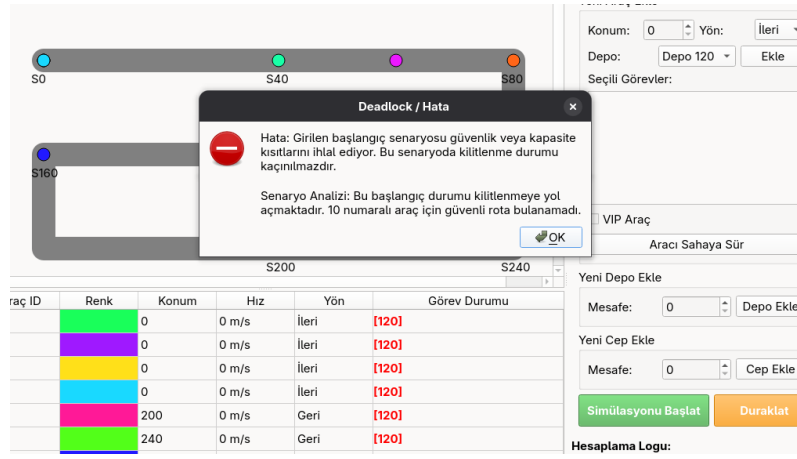


Şekil 17: Dinamik Başlangıç Durumu

4.5.2 Analiz

Bölüm 3.3.4’de bahsedilen çizelgelemeye göre, önce çizelgelenen araç hedefine en yakın olan $x = 120$ konumunda başlayan A11 aracıdır. Bu aracın bulunduğu konumdaki depoya girdikten sonra doğrudan başlangıç konumuna yönelmektedir. Bu sırada yolundaki araçlardan A10 aracı bulunduğu konumdaki cebe girerken A8 ve A9 araçları ise yol verecek bir yer olmamasından ötürü başlangıç konumuna gitmektedir.

Ceplerin Kaldırılması C60 ve C220 ceplerinin kaldırılması durumunda ise program güvenli mesafe kuralının sağlanamaması durumundan ve rota bulunamamasından ötürü kilitlenme uyarısı vermektedir Şekil 18.



Şekil 18: Kilitlenme Uyarısı

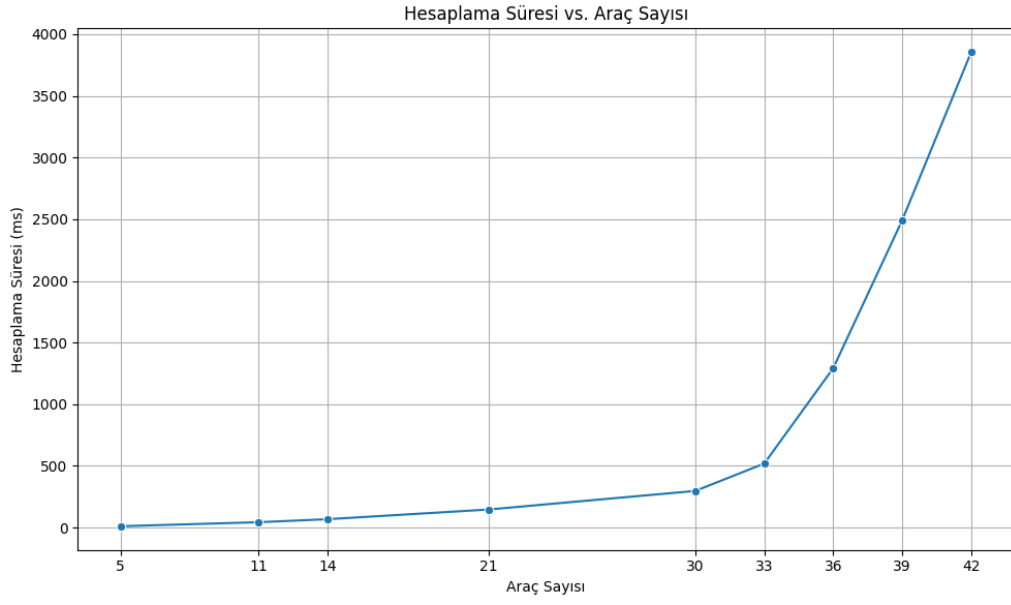
4.6 Performans Analizi

Analizi yapılan senaryolar ardından sistem Bölüm 4.3'teki tek görevli senaryo baz alınarak farklı araç sayılarındaki hesaplama performansı test edilmiştir.

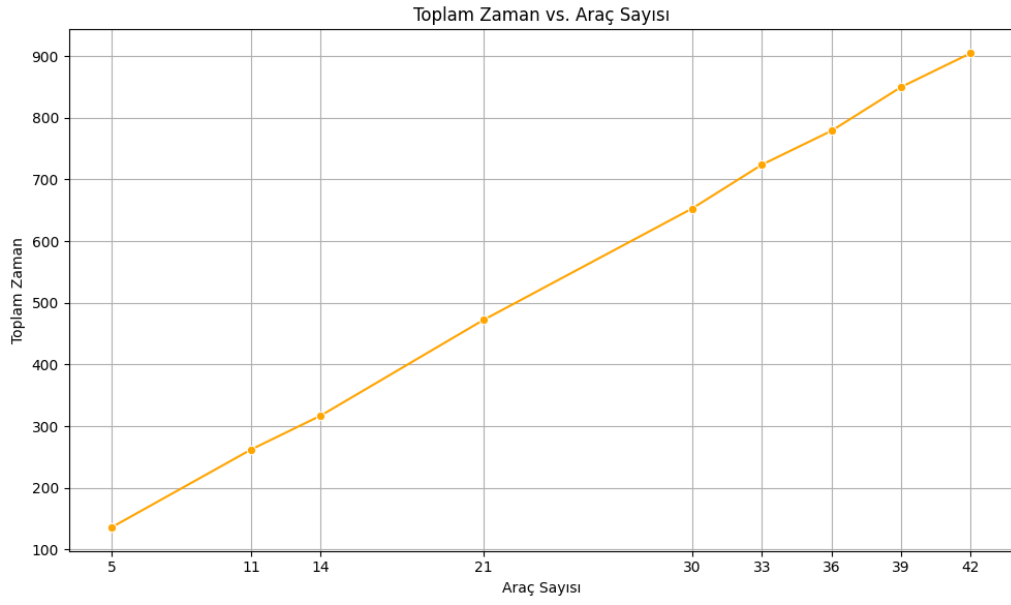
Tablo 4: Sonuçlar

Araç Sayısı	Hesaplama Süresi (ms)	Toplam Zaman
5	11.1	136
11	43.37	262
14	68.0	317
21	146.32	472
30	297.04	653
33	518.83	724
36	1291.78	779
39	2490.93	850
42	3854.37	905

Şekil 19'de gözüktüğü üzere hesaplama zamanı araç sayısı arttıkça üstel olarak artmaktadır. Toplam görev tamamlanma zamanıysa (Şekil 20) araç sayısına bağlı olarak doğrusal bir artış eğilimindedir.



Şekil 19: Hesaplama Süresi - Araç Sayısı



Şekil 20: Toplam Zaman - Araç Sayısı

5 Sonuçlar ve Öneriler

Bu çalışma kapsamında, tek şeritli ve dar koridor topolojisine sahip endüstriyel AGV hatlarında karşılaşılan kilitlenme (*deadlock*) ve darboğaz problemlerini çözmek amacıyla, klasik İşbirlikçi A* (Cooperative A*) algoritması üzerinde kural ve öncelik tabanlı melez bir model geliştirilmiş ve başarıyla gerçekleştirilmiştir. Elde edilen bulgular, yapılan senaryo analizleri ve performans testleri doğrultusunda sistemin çıktıları aşağıda detaylandırılmıştır.

5.1 Sonuçlar

Geliştirilen melez planlama yaklaşımının teorik ve pratik kazanımları üç ana başlık altında toplanmaktadır:

- 1. Kilitlenmelerin Önlenmesi ve Rota Kararlılığı:** Klasik CA* algoritmasının tek şeritli hatlardaki en belirgin zafiyeti olan koridor kilitlenmeleri, bu çalışmada önerilen dört bileşenli sözlüksel öncelik vektörü ρ_i , dinamik gecikmeli çıkış (*Delayed Release*) mekanizması ve bekleme eylemli kısıtlamasıyla aşılmıştır. Yapılan yoğun hat sınamalarında, fiziksel güvenlik mesafesi kısıtının ($d_{min} = 20$ m) anlık olarak ihlal edildiği durumlar dışında, algoritmik veya yapısal herhangi bir kilitlenme ya da kafa kafaya çarpışma hatasıyla karşılaşmamıştır. Bu durum, kısıt yönetiminin dar alanlarda deterministik bir kararlılık sunduğunu kanıtlamaktadır.
- 2. Asimetrik Maliyet ile Git Gel Hareketlerinin Önlenmesi:** Araçların transit hat üzerinde durmasının yasaklanması ve bekleme eylemlerinin kapasite kısıtlı cepler ile depolarla sınırlandırılması, asimetrik maliyet fonksiyonuyla desteklenmiştir. Hareket maliyetinin 1.0, güvenli noktalarda bekleme maliyetinin ise 0.9 olarak atanması sayesinde araçların rotayı uzatan gereksiz ileri-geri (*yo-yo*) manevralar yapması matematiksel olarak engellenmiştir. Simülasyonlarda araçların ceplerde veya depolarda verimli bir şekilde beklediği ve ana transit hattı gereksiz yere işgal etmediği gözlemlenmiştir.
- 3. Darboğaz Yönetimi ve Erken Tahliye Başarısı:** Çoklu istasyon ziyareti yapan araçların görev hatları En Kısa İş İlk (SJF) yaklaşımına göre yakından uzağa sıralanmıştır. Darboğaz sınamalarında, tüm araçların aynı uzak depoya (D240) yöneldiği senaryoda hesaplama süresi artsa da toplam tamamlanma süresinin (*makespan*) 54 adım azaldığı tespit edilmiştir. Bu durum, SJF stratejisinin kısa mesafeli görevleri hızla eriterek ana yolu erken boşalttığını ve uzay-zaman matrisindeki rezervasyon çakışmalarını minimize ettiğini somut olarak göstermektedir.

Sistemin ölçeklenebilirlik sınırlarını belirlemek adına yapılan performans çözümlemelerinde, araç sayısının 5'ten 42'ye kadar çıkarıldığı senaryolar sınanmıştır. Elde edilen çıktılarına göre toplam görev tamamlanma zamanı (*makespan*) araç sayısına bağlı olarak doğrusal bir artış eğilimi gösterirken hesaplama süresinin üstel olarak büyüdüğü gözlemlenmiştir. Bu üstel büyümenin temel nedeni hat yoğunluğu arttıkça kilitlenme çözücü (*deadlock resolver*) mekanizmasının daha sık devreye girerek yola çıkış zamanlarını ötelemesi ve A* aramasını tekrarlatmasıdır. Ancak AGV yönlendirme problemlerinin doğası gereği bu hesaplamalar araçlar yola çıkmadan önce çevrim dışı planlama aşamasında yapıldığından 42 araçta ulaşılan yaklaşık 3.85 saniyelik (3854.37 ms) işlem süresi gerçek hayat pratikleri ve güvenli operasyon sürdürülebilirliği açısından tamamen kabul edilebilir bir düzeydedir.

5.2 Öneriler ve Gelecek Çalışmalar

Geliştirilen modelin mevcut başarımlarını daha ileriye taşımak ve tespit edilen zamansal sınırlılıkları ortadan kaldırmak adına gelecek çalışmalarda şu yaklaşımların uygulanması önerilmektedir:

- **Bölgesel ve Dağıtık (*Decentralized*) Planlama:** Araç sayısı arttıkça merkezi rezervasyon tablosunun yönetiminden ve kilitlenme çözücü tekrarlarından kaynaklanan üstel hesaplama maliyetini düşürmek amacıyla sistem dağıtık bir yapıya dönüştürülebilir. Araçların yalnızca kendi çevrelerindeki belirli bir algılama/iletişim yarıçapı içerisindeki alanları tahsis ettiği bölgesel planlama yaklaşımları incelenerek algoritmik zaman karmaşıklığı iyileştirilebilir.
- **Dinamik Hat ve Pencere Rezervasyonu:** Tüm rotayı en baştan rezerve etmek yerine, araçların hareket ettikçe zaman içinde kayan zaman pencereleri (*rolling-horizon / sliding time windows*) üzerinden dinamik rezervasyon yapması sağlanabilir. Bu sayede uzay-zaman matrisinin yoğunluğu azaltılarak yapay kilitlenmelerin (*artificial deadlocks*) ve buna bağlı gecikmeli çıkış ötelemelerinin önüne geçilebilir.
- **Kapasite Değişimleri ve Öğrenme Tabanlı Entegrasyon:** Güvenli ceplerin kapasitelerinin dinamik olarak değiştirilmesi veya yoğun hatlar üzerinde hangi aracın hangi cebe ne kadar süreyle çekileceğinin takviyeli öğrenme (*Reinforcement Learning*) modelleriyle tahmin edilmesi, kurallı melez mimarinin esnekliğini artırabilir. Gelecek çalışmalarda bu tür öğrenme tabanlı yaklaşımlarla kural tabanlı kısıtların hibrit entegrasyonu hedeflenmektedir.

Kaynaklar

- Ma, H., Harabor, D., Stuckey, P. J., Li, J., and Koenig, S. (2019). Searching with Consistent Prioritization for Multi-Agent Path Finding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01):7643–7650.
- Okumura, K., Machida, M., Défago, X., and Tamura, Y. (2022). Priority inheritance with backtracking for iterative multi-agent path finding. *Artificial Intelligence*, 310:103752.
- Sajid, Q., Luna, R., and Bekris, K. (2021). Multi-Agent Pathfinding with Simultaneous Execution of Single-Agent Primitives. *Proceedings of the International Symposium on Combinatorial Search*, 3(1):88–96.
- Sharon, G., Stern, R., Felner, A., and Sturtevant, N. R. (2015). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66.
- Silver, D. (2005). Cooperative Pathfinding. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 1(1):117–122.
- Surynek, P. (2022). Problem Compilation for Multi-Agent Path Finding: A Survey. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, pages 5615–5622, Vienna, Austria. International Joint Conferences on Artificial Intelligence Organization.
- Wang, H. and Rubenstein, M. (2020). Walk, Stop, Count, and Swap: Decentralized Multi-Agent Path Finding With Theoretical Guarantees. *IEEE Robotics and Automation Letters*, 5(2):1119–1126.
- Wu, M., Yan, W., Hasan, H., and Jamaluddin, R. A. (2023). A Review of Multi-Agent Path Finding Algorithms. In *2023 11th International Conference on Information Systems and Computing Technology (ISCTech)*, pages 69–73, Qingdao, China. IEEE.